La Taberna del Bus de Eventos

"El Gólem Alquímico"

Patrón arquitectónico Publisher/Subscriber con Event Bus en Java

Diagramas UML de clases, secuencia y componentes (PlantUML)

Mia Montserrat Espinoza Rosales

Matrícula 1199574

Ingeniería en Computación

Universidad Autónoma de Baja California, campus Mexicali

2 de noviembre de 2026

1. Descripción de la arquitectura

La taberna se rediseñó con un **Event Bus** para desacoplar todos sus componentes. Los componentes **publican** eventos inmutables en el bus y otros componentes se **suscriben** a los tipos de evento que les interesan. Ningún componente conoce a otro: todos dependen únicamente de la interfaz `BusDeEventos`, de la interfaz `Suscriptor` y de las clases de eventos.

Eventos

`PedidoRealizadoEvent`, `BebidaServidaEvent`, `ComidaPreparadaEvent` y `BandaTocandoEvent`. Son records de Java: inmutables, validados en el constructor y con copia defensiva de las listas.

Componentes

Publicadores: SistemaDePedidos y Banda. **Suscriptores y publicadores:** Barra y Cocina (2 s por bebida y 5 s por platillo, con un planificador de tareas que no bloquea el bus). **Suscriptores:** SistemaDeSonido y PanelLED (suscrito a todos los eventos).

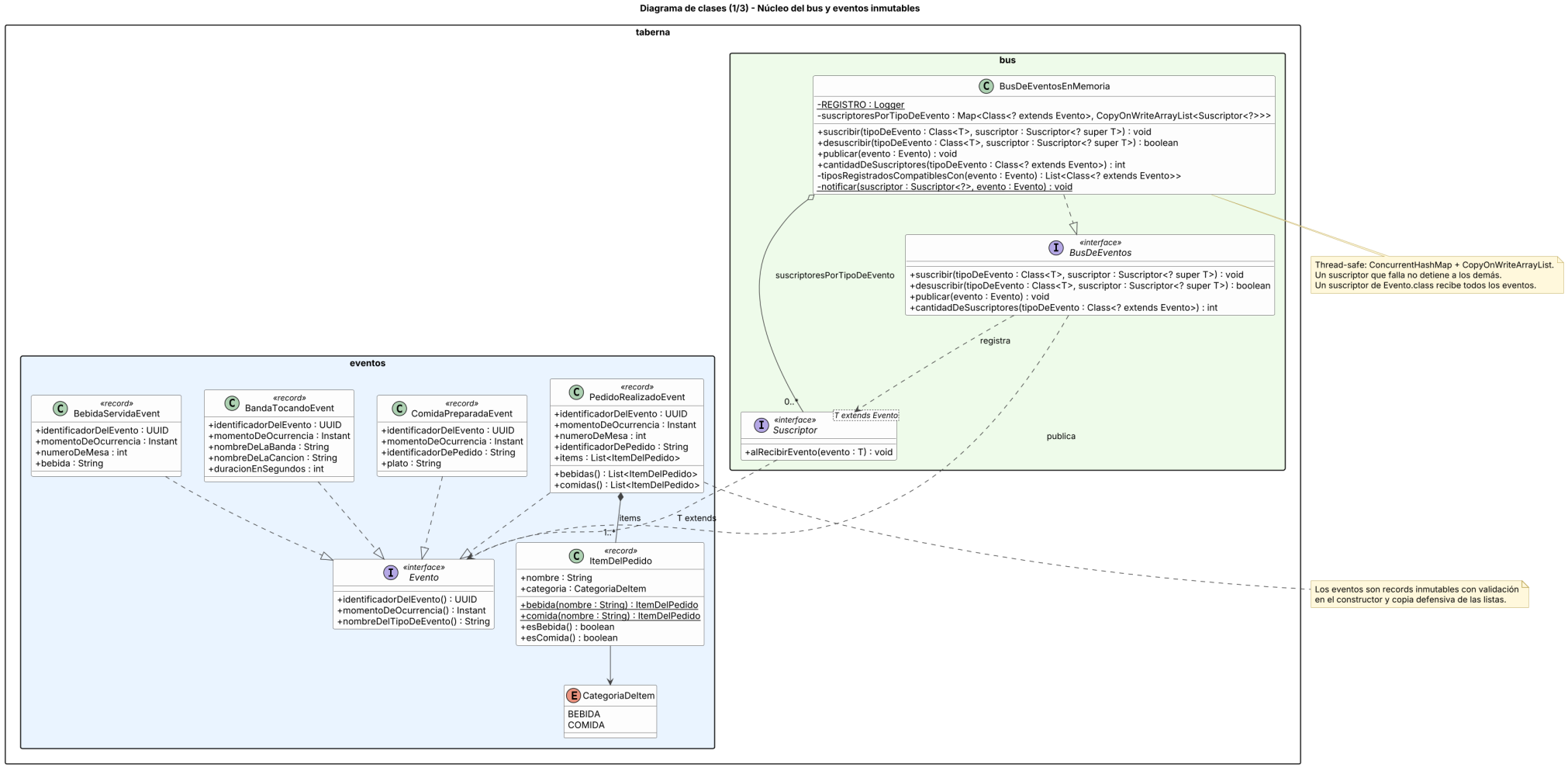
Extensibilidad (Open/Closed)

SistemaDeHumo, SistemaDeIluminacion y RegistroDeAuditoria se agregan en la clase `DemostracionDeExtensibilidad` sin modificar ni un carácter de la Banda, del bus ni de los demás componentes.

Contenido de este documento

2. Diagramas de clases (3 partes) 3. Diagrama de secuencia 4. Diagrama de componentes. Cada diagrama incluye el código PlantUML con el que fue generado.

2.1 Diagrama de clases: núcleo del bus y eventos



Código PlantUML - núcleo del bus y eventos

```
@startuml 01a-clases-bus-y-eventos
title Diagrama de clases (1/3) - Núcleo del bus y eventos inmutables

skinparam dpi 130
skinparam shadowing false
skinparam classAttributeIconSize 0
skinparam packageStyle rectangle
skinparam class {
  BackgroundColor #FDFDFD
  BorderColor #444444
  ArrowColor #444444
}
skinparam note {
  BackgroundColor #FFF8DC
  BorderColor #B8A04A
}

package "taberna.bus" #EEFBEA {
  interface Suscriptor<T extends Evento> <<interface>> {
    +alRecibirEvento(evento : T) : void
  }

  interface BusDeEventos <<interface>> {
    +suscribir(tipoDeEvento : Class<T>, suscriptor : Suscriptor<? super T>) : void
    +desuscribir(tipoDeEvento : Class<T>, suscriptor : Suscriptor<? super T>) : boolean
    +publicar(evento : Evento) : void
    +cantidadDeSuscriptores(tipoDeEvento : Class<? extends Evento>) : int
  }

  class BusDeEventosEnMemoria {
    -REGISTRO : Logger {static}
    -suscriptoresPorTipoDeEvento : Map<Class<? extends Evento>, CopyOnWriteArrayList<Suscriptor<?>>>
    +suscribir(tipoDeEvento : Class<T>, suscriptor : Suscriptor<? super T>) : void
    +desuscribir(tipoDeEvento : Class<T>, suscriptor : Suscriptor<? super T>) : boolean
    +publicar(evento : Evento) : void
    +cantidadDeSuscriptores(tipoDeEvento : Class<? extends Evento>) : int
    -tiposRegistradosCompatiblesCon(evento : Evento) : List<Class<? extends Evento>>
    -notificar(suscriptor : Suscriptor<?>, evento : Evento) : void {static}
  }
}

package "taberna.eventos" #EAF4FF {
  interface Evento <<interface>> {
    +identificadorDelEvento() : UUID
    +momentoDeOcurrencia() : Instant
    +nombreDelTipoDeEvento() : String
  }

  enum CategoriaDeItem {
    BEBIDA
    COMIDA
  }

  class ItemDelPedido <<record>> {
    +nombre : String
    +categoria : CategoriaDeItem
    +bebida(nombre : String) : ItemDelPedido {static}
    +comida(nombre : String) : ItemDelPedido {static}
    +esBebida() : boolean
    +esComida() : boolean
  }

  class PedidoRealizadoEvent <<record>> {
    +identificadorDelEvento : UUID
    +momentoDeOcurrencia : Instant
    +numeroDeMesa : int
    +identificadorDePedido : String
    +items : List<ItemDelPedido>
    +bebidas() : List<ItemDelPedido>
    +comidas() : List<ItemDelPedido>
  }

  class BebidaServidaEvent <<record>> {
    +identificadorDelEvento : UUID
    +momentoDeOcurrencia : Instant
    +numeroDeMesa : int
    +bebida : String
  }

  class ComidaPreparadaEvent <<record>> {
    +identificadorDelEvento : UUID
    +momentoDeOcurrencia : Instant
    +identificadorDePedido : String
    +plato : String
  }

  class BandaTocandoEvent <<record>> {
    +identificadorDelEvento : UUID
    +momentoDeOcurrencia : Instant
    +nombreDeLaBanda : String
    +nombreDeLaCancion : String
    +duracionEnSegundos : int
  }
}

BusDeEventosEnMemoria ..|> BusDeEventos
BusDeEventosEnMemoria o-- "0..*" Suscriptor : suscriptoresPorTipoDeEvento
BusDeEventos ..> Suscriptor : registra
BusDeEventos ..> Evento : publica
Suscriptor ..> Evento : T extends

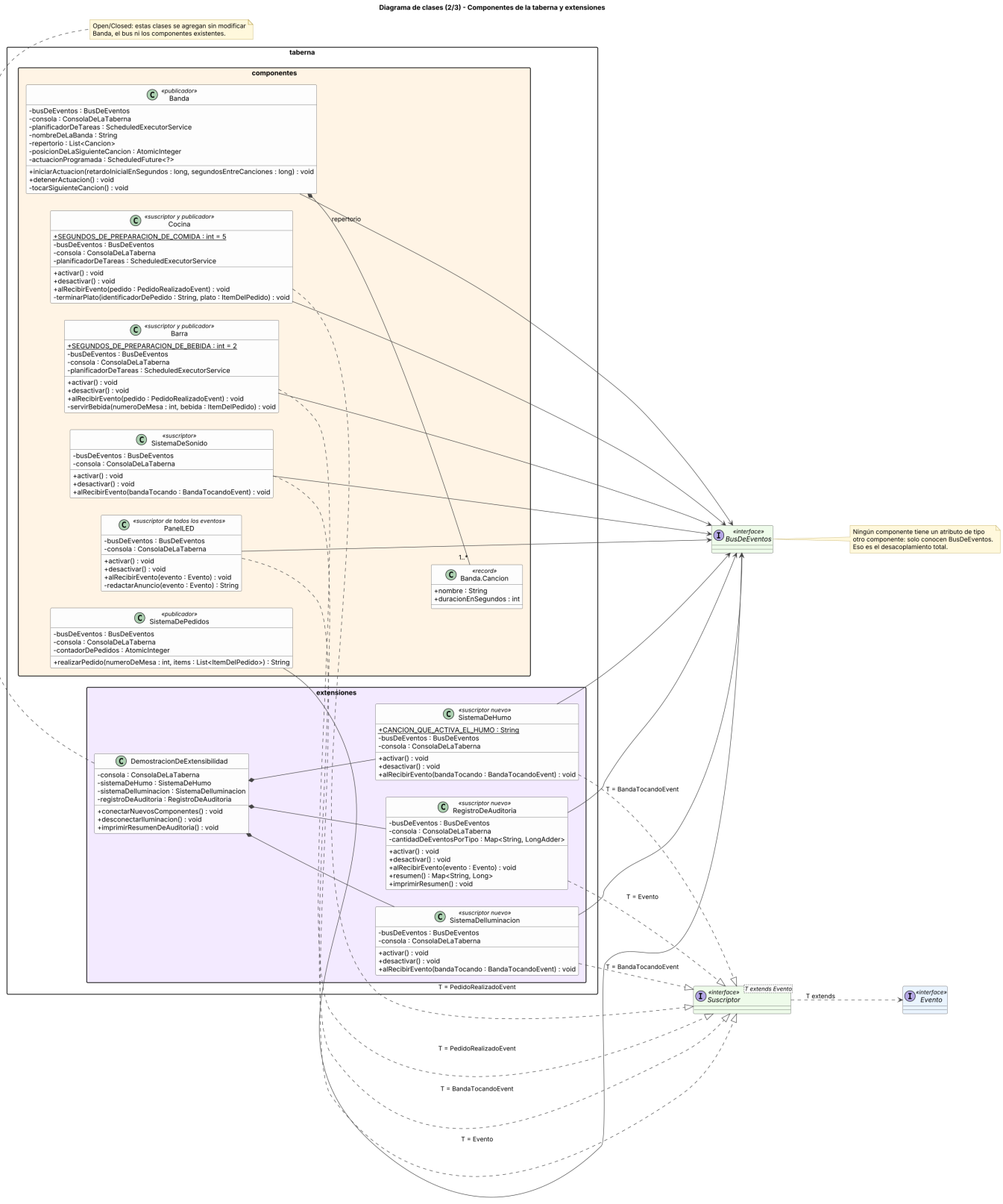
PedidoRealizadoEvent ..|> Evento
BebidaServidaEvent ..|> Evento
ComidaPreparadaEvent ..|> Evento
BandaTocandoEvent ..|> Evento
```

```
PedidoRealizadoEvent *-- "1..*" ItemDelPedido : items
ItemDelPedido --> CategoriaDeItem

note bottom of BusDeEventosEnMemoria
  Thread-safe: ConcurrentHashMap + CopyOnWriteArrayList.
  Un suscriptor que falla no detiene a los demás.
  Un suscriptor de Evento.class recibe todos los eventos.
end note

note bottom of PedidoRealizadoEvent
  Los eventos son records inmutables con validación
  en el constructor y copia defensiva de las listas.
end note
@enduml
```

2.2 Diagrama de clases: componentes y extensiones



Código PlantUML - componentes y extensiones

```
@startuml 01b-clases-componentes
title Diagrama de clases (2/3) - Componentes de la taberna y extensiones

skinparam dpi 130
skinparam shadowing false
skinparam classAttributeIconSize 0
skinparam packageStyle rectangle
left to right direction
skinparam class {
  BackgroundColor #FDFDFD
  BorderColor #444444
  ArrowColor #444444
}
skinparam note {
  BackgroundColor #FFF8DC
  BorderColor #B8A04A
}

' ---- Abstracciones (detalladas en el diagrama 1/3) ----
interface BusDeEventos <<interface>> #EEFBEA
interface "Suscriptor<T extends Evento>" as Suscriptor <<interface>> #EEFBEA
interface Evento <<interface>> #EAF4FF

package "taberna.componentes" #FFF4E5 {
  class SistemaDePedidos <<publicador>> {
    -busDeEventos : BusDeEventos
    -consola : ConsolaDeLaTaberna
    -contadorDePedidos : AtomicInteger
    +realizarPedido(numeroDeMesa : int, items : List<ItemDelPedido>) : String
  }

  class Barra <<suscriptor y publicador>> {
    +SEGUNDOS_DE_PREPARACION_DE_BEBIDA : int = 2 {static}
    -busDeEventos : BusDeEventos
    -consola : ConsolaDeLaTaberna
    -planificadorDeTareas : ScheduledExecutorService
    +activar() : void
    +desactivar() : void
    +alRecibirEvento(pedido : PedidoRealizadoEvent) : void
    -servirBebida(numeroDeMesa : int, bebida : ItemDelPedido) : void
  }

  class Cocina <<suscriptor y publicador>> {
    +SEGUNDOS_DE_PREPARACION_DE_COMIDA : int = 5 {static}
    -busDeEventos : BusDeEventos
    -consola : ConsolaDeLaTaberna
    -planificadorDeTareas : ScheduledExecutorService
    +activar() : void
    +desactivar() : void
    +alRecibirEvento(pedido : PedidoRealizadoEvent) : void
    -terminarPlato(identificadorDePedido : String, plato : ItemDelPedido) : void
  }

  class Banda <<publicador>> {
    -busDeEventos : BusDeEventos
    -consola : ConsolaDeLaTaberna
    -planificadorDeTareas : ScheduledExecutorService
    -nombreDeLaBanda : String
    -repertorio : List<Cancion>
    -posicionDeLaSiguienteCancion : AtomicInteger
    -actuacionProgramada : ScheduledFuture<?>
    +iniciarActuacion(retardoInicialEnSegundos : long, segundosEntreCanciones : long) : void
    +detenerActuacion() : void
    -tocarSiguienteCancion() : void
  }

  class "Banda.Cancion" as Cancion <<record>> {
    +nombre : String
    +duracionEnSegundos : int
  }

  class SistemaDeSonido <<suscriptor>> {
    -busDeEventos : BusDeEventos
    -consola : ConsolaDeLaTaberna
    +activar() : void
    +desactivar() : void
    +alRecibirEvento(bandaTocando : BandaTocandoEvent) : void
  }

  class PanelLED <<suscriptor de todos los eventos>> {
    -busDeEventos : BusDeEventos
    -consola : ConsolaDeLaTaberna
    +activar() : void
    +desactivar() : void
    +alRecibirEvento(evento : Evento) : void
    -redactarAnuncio(evento : Evento) : String
  }
}

package "taberna.extensiones" #F3EAFF {
  class SistemaDeHumo <<suscriptor nuevo>> {
    +CANCION_QUE_ACTIVA_EL_HUMO : String {static}
    -busDeEventos : BusDeEventos
    -consola : ConsolaDeLaTaberna
    +activar() : void
    +desactivar() : void
    +alRecibirEvento(bandaTocando : BandaTocandoEvent) : void
  }

  class SistemaDeIluminacion <<suscriptor nuevo>> {
    -busDeEventos : BusDeEventos
    -consola : ConsolaDeLaTaberna
    +activar() : void
    +desactivar() : void
    +alRecibirEvento(bandaTocando : BandaTocandoEvent) : void
```

```
  }

  class RegistroDeAuditoria <<suscriptor nuevo>> {
    -busDeEventos : BusDeEventos
    -consola : ConsolaDeLaTaberna
    -cantidadDeEventosPorTipo : Map<String, LongAdder>
    +activar() : void
    +desactivar() : void
    +alRecibirEvento(evento : Evento) : void
    +resumen() : Map<String, Long>
    +imprimirResumen() : void
  }

  class DemostracionDeExtensibilidad {
    -consola : ConsolaDeLaTaberna
    -sistemaDeHumo : SistemaDeHumo
    -sistemaDeIluminacion : SistemaDeIluminacion
    -registroDeAuditoria : RegistroDeAuditoria
    +conectarNuevosComponentes() : void
    +desconectarIluminacion() : void
    +imprimirResumenDeAuditoria() : void
  }
}

' ---- Implementan Suscriptor ----
Barra ..|> Suscriptor : T = PedidoRealizadoEvent
Cocina ..|> Suscriptor : T = PedidoRealizadoEvent
SistemaDeSonido ..|> Suscriptor : T = BandaTocandoEvent
SistemaDeHumo ..|> Suscriptor : T = BandaTocandoEvent
SistemaDeIluminacion ..|> Suscriptor : T = BandaTocandoEvent
PanelLED ..|> Suscriptor : T = Evento
RegistroDeAuditoria ..|> Suscriptor : T = Evento

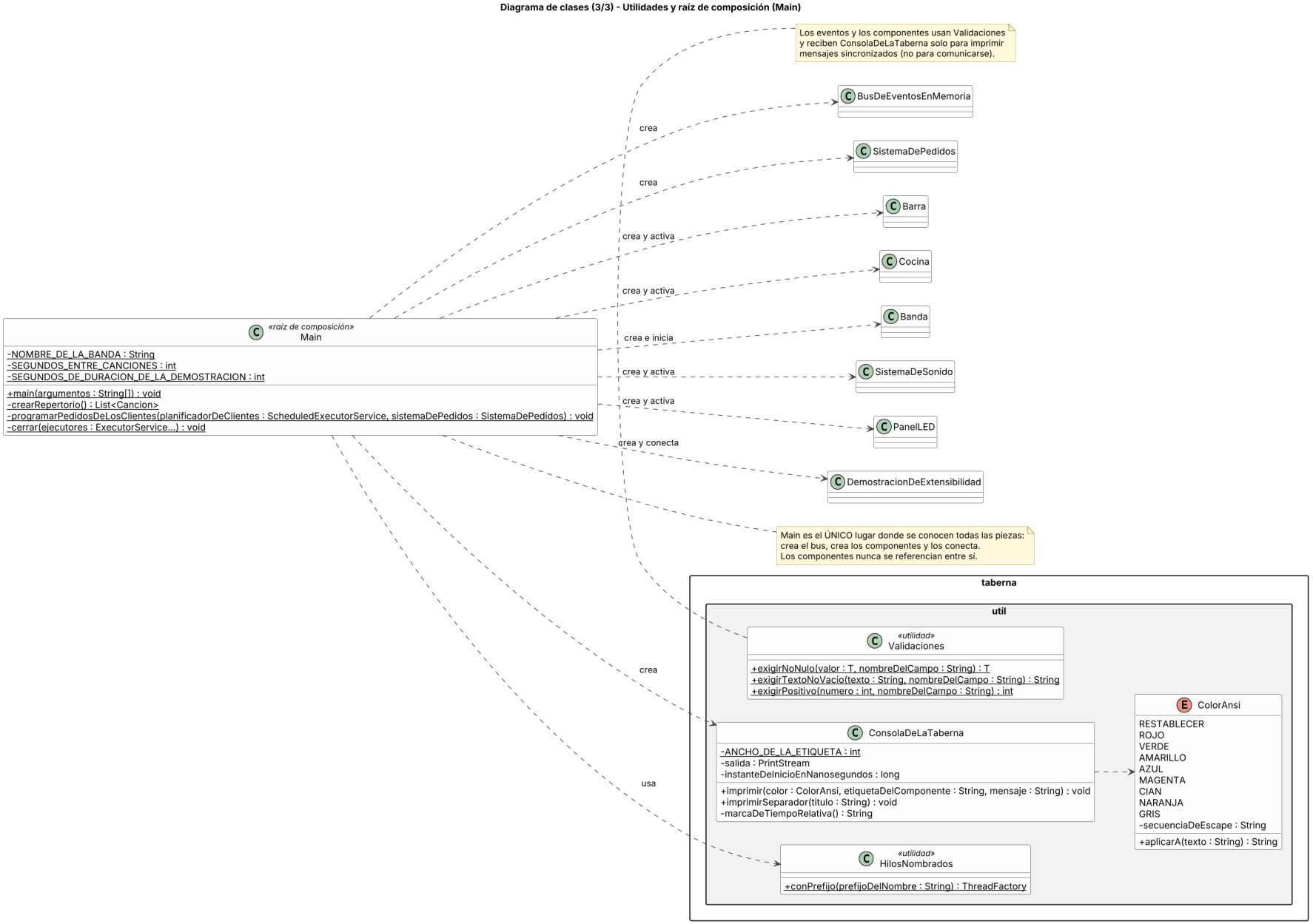
' ---- Cada componente SOLO conoce la abstracción del bus ----
SistemaDePedidos --> BusDeEventos
Barra --> BusDeEventos
Cocina --> BusDeEventos
Banda --> BusDeEventos
SistemaDeSonido --> BusDeEventos
PanelLED --> BusDeEventos
SistemaDeHumo --> BusDeEventos
SistemaDeIluminacion --> BusDeEventos
RegistroDeAuditoria --> BusDeEventos

Banda *-- "1..*" Cancion : repertorio
DemostracionDeExtensibilidad *-- SistemaDeHumo
DemostracionDeExtensibilidad *-- SistemaDeIluminacion
DemostracionDeExtensibilidad *-- RegistroDeAuditoria
Suscriptor ..> Evento : T extends

note right of BusDeEventos
  Ningún componente tiene un atributo de tipo
  otro componente: solo conocen BusDeEventos.
  Eso es el desacoplamiento total.
end note

note bottom of DemostracionDeExtensibilidad
  Open/Closed: estas clases se agregan sin modificar
  Banda, el bus ni los componentes existentes.
end note
@enduml
```

2.3 Diagrama de clases: utilidades y raíz de composición (Main)



Código PlantUML - utilidades y raíz de composición (Main)

```
@startuml 01c-clases-utilidades-y-main
title Diagrama de clases (3/3) - Utilidades y raíz de composición (Main)

skinparam dpi 130
skinparam shadowing false
skinparam classAttributeIconSize 0
skinparam packageStyle rectangle
left to right direction
skinparam class {
  BackgroundColor #FDFDFD
  BorderColor #444444
  ArrowColor #444444
}
skinparam note {
  BackgroundColor #FFF8DC
  BorderColor #B8A04A
}

package "taberna.util" #F2F2F2 {
  class ConsolaDeLaTaberna {
    -ANCHO_DE_LA_ETIQUETA : int {static}
    -salida : PrintStream
    -instanteDeInicioEnNanosegundos : long
    +imprimir(color : ColorAnsi, etiquetaDelComponente : String, mensaje : String) : void
    +imprimirSeparador(titulo : String) : void
    -marcaDeTiempoRelativa() : String
  }

  enum ColorAnsi {
    RESTABLECER
    ROJO
    VERDE
    AMARILLO
    AZUL
    MAGENTA
    CIAN
    NARANJA
    GRIS
    -secuenciaDeEscape : String
    +aplicarA(texto : String) : String
  }

  class Validaciones <<utilidad>> {
    +exigirNoNulo(valor : T, nombreDelCampo : String) : T {static}
    +exigirTextoNoVacio(texto : String, nombreDelCampo : String) : String {static}
    +exigirPositivo(numero : int, nombreDelCampo : String) : int {static}
  }

  class HilosNombrados <<utilidad>> {
    +conPrefijo(prefijoDelNombre : String) : ThreadFactory {static}
  }
}

class Main <<raíz de composición>> {
  -NOMBRE_DE_LA_BANDA : String {static}
  -SEGUNDOS_ENTRE_CANCIONES : int {static}
  -SEGUNDOS_DE_DURACION_DE_LA_DEMOSTRACION : int {static}
  +main(argumentos : String[]) : void {static}
  -crearRepertorio() : List<Cancion> {static}
  -programarPedidosDeLosClientes(planificadorDeClientes : ScheduledExecutorService, sistemaDePedidos : SistemaDePedidos) : void {static}
  -cerrar(ejecutores : ExecutorService...) : void {static}
}

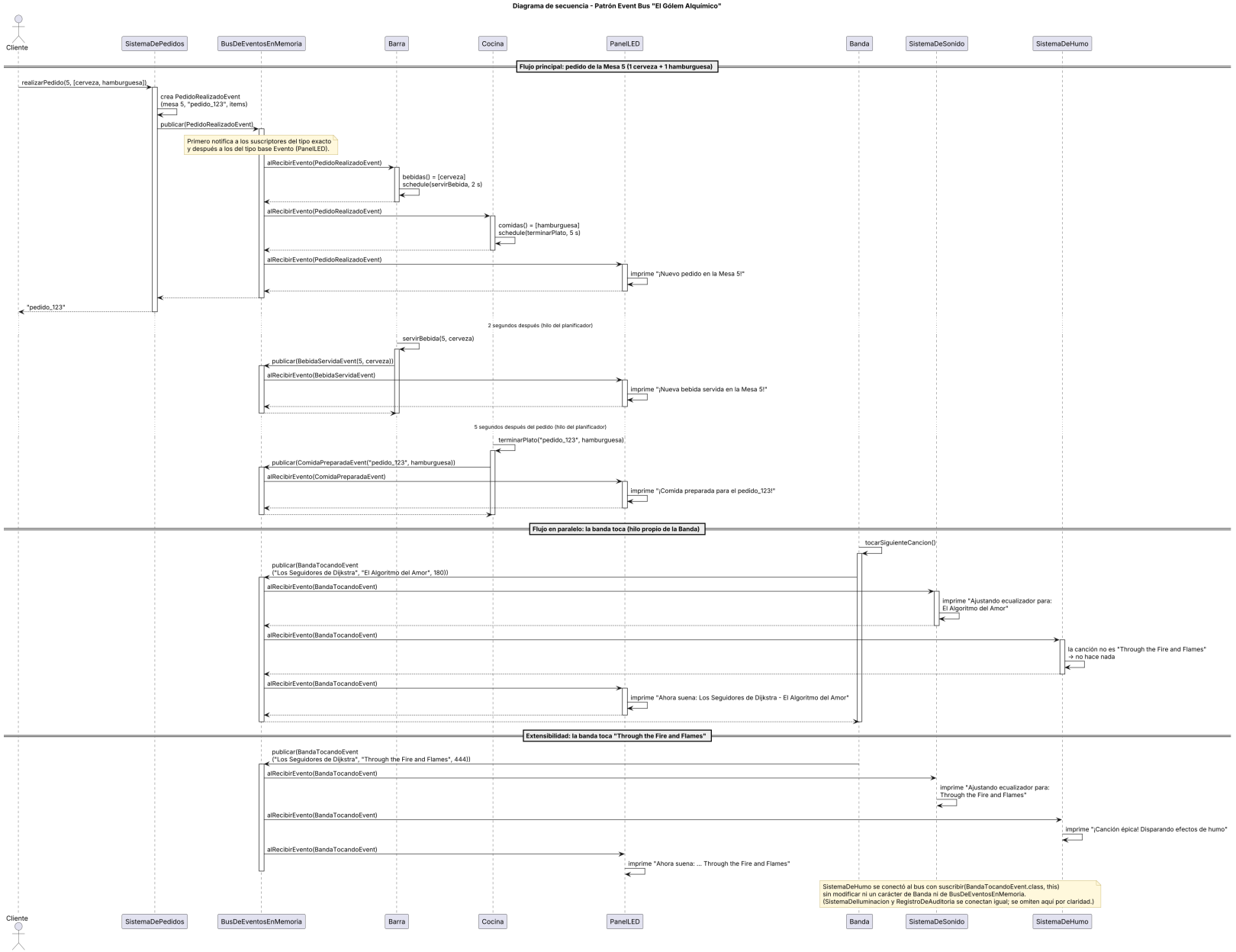
class BusDeEventosEnMemoria
class SistemaDePedidos
class Barra
class Cocina
class Banda
class SistemaDeSonido
class PanelLED
class DemostracionDeExtensibilidad

ConsolaDeLaTaberna ..> ColorAnsi
Main ..> ConsolaDeLaTaberna : crea
Main ..> HilosNombrados : usa
Main ..> BusDeEventosEnMemoria : crea
Main ..> SistemaDePedidos : crea
Main ..> Barra : crea y activa
Main ..> Cocina : crea y activa
Main ..> Banda : crea e inicia
Main ..> SistemaDeSonido : crea y activa
Main ..> PanelLED : crea y activa
Main ..> DemostracionDeExtensibilidad : crea y conecta

note right of Main
  Main es el ÚNICO lugar donde se conocen todas las piezas:
  crea el bus, crea los componentes y los conecta.
  Los componentes nunca se referencian entre sí.
end note

note bottom of Validaciones
  Los eventos y los componentes usan Validaciones
  y reciben ConsolaDeLaTaberna solo para imprimir
  mensajes sincronizados (no para comunicarse).
end note
@enduml
```

3. Diagrama de secuencia: pedido de la Mesa 5 y banda tocando



Código PlantUML - diagrama de secuencia

```
@startuml 02-diagrama-de-secuencia
title Diagrama de secuencia - Patrón Event Bus "El Gólem Alquímico"

skinparam dpi 130
skinparam shadowing false
skinparam sequenceMessageAlign left
skinparam responseMessageBelowArrow true
skinparam noteBackgroundColor #FFF8DC
skinparam noteBorderColor #B8A04A
skinparam ParticipantPadding 12

actor Cliente
participant "SistemaDePedidos" as Pedidos
participant "BusDeEventosEnMemoria" as Bus
participant "Barra" as Barra
participant "Cocina" as Cocina
participant "PanelLED" as Panel
participant "Banda" as Banda
participant "SistemaDeSonido" as Sonido
participant "SistemaDeHumo" as Humo

== Flujo principal: pedido de la Mesa 5 (1 cerveza + 1 hamburguesa) ==

Cliente -> Pedidos : realizarPedido(5, [cerveza, hamburguesa])
activate Pedidos
Pedidos -> Pedidos : crea PedidoRealizadoEvent\n(mesa 5, "pedido_123", items)
Pedidos -> Bus : publicar(PedidoRealizadoEvent)
activate Bus
note over Bus
  Primero notifica a los suscriptores del tipo exacto
  y después a los del tipo base Evento (PanelLED).
end note
Bus -> Barra : alRecibirEvento(PedidoRealizadoEvent)
activate Barra
Barra -> Barra : bebidas() = [cerveza]\nschedule(servirBebida, 2 s)
Barra --> Bus
deactivate Barra
Bus -> Cocina : alRecibirEvento(PedidoRealizadoEvent)
activate Cocina
Cocina -> Cocina : comidas() = [hamburguesa]\nschedule(terminarPlato, 5 s)
Cocina --> Bus
deactivate Cocina
Bus -> Panel : alRecibirEvento(PedidoRealizadoEvent)
activate Panel
Panel -> Panel : imprime "¡Nuevo pedido en la Mesa 5!"
Panel --> Bus
deactivate Panel
Bus --> Pedidos
deactivate Bus
Pedidos --> Cliente : "pedido_123"
deactivate Pedidos

... 2 segundos después (hilo del planificador) ...

Barra -> Barra : servirBebida(5, cerveza)
activate Barra
Barra -> Bus : publicar(BebidaServidaEvent(5, cerveza))
activate Bus
Bus -> Panel : alRecibirEvento(BebidaServidaEvent)
activate Panel
Panel -> Panel : imprime "¡Nueva bebida servida en la Mesa 5!"
Panel --> Bus
deactivate Panel
Bus --> Barra
deactivate Bus
deactivate Barra

... 5 segundos después del pedido (hilo del planificador) ...

Cocina -> Cocina : terminarPlato("pedido_123", hamburguesa)
activate Cocina
Cocina -> Bus : publicar(ComidaPreparadaEvent("pedido_123", hamburguesa))
activate Bus
Bus -> Panel : alRecibirEvento(ComidaPreparadaEvent)
activate Panel
Panel -> Panel : imprime "¡Comida preparada para el pedido_123!"
Panel --> Bus
deactivate Panel
Bus --> Cocina
deactivate Bus
deactivate Cocina

== Flujo en paralelo: la banda toca (hilo propio de la Banda) ==

Banda -> Banda : tocarSiguienteCancion()
activate Banda
Banda -> Bus : publicar(BandaTocandoEvent\n("Los Seguidores de Dijkstra", "El Algoritmo del Amor", 180))
activate Bus
Bus -> Sonido : alRecibirEvento(BandaTocandoEvent)
activate Sonido
Sonido -> Sonido : imprime "Ajustando ecualizador para:\nEl Algoritmo del Amor"
Sonido --> Bus
deactivate Sonido
Bus -> Humo : alRecibirEvento(BandaTocandoEvent)
activate Humo
Humo -> Humo : la canción no es "Through the Fire and Flames"\n→ no hace nada
Humo --> Bus
deactivate Humo
Bus -> Panel : alRecibirEvento(BandaTocandoEvent)
activate Panel
Panel -> Panel : imprime "Ahora suena: Los Seguidores de Dijkstra - El Algoritmo del Amor"
Panel --> Bus
deactivate Panel
Bus --> Banda
deactivate Bus
```

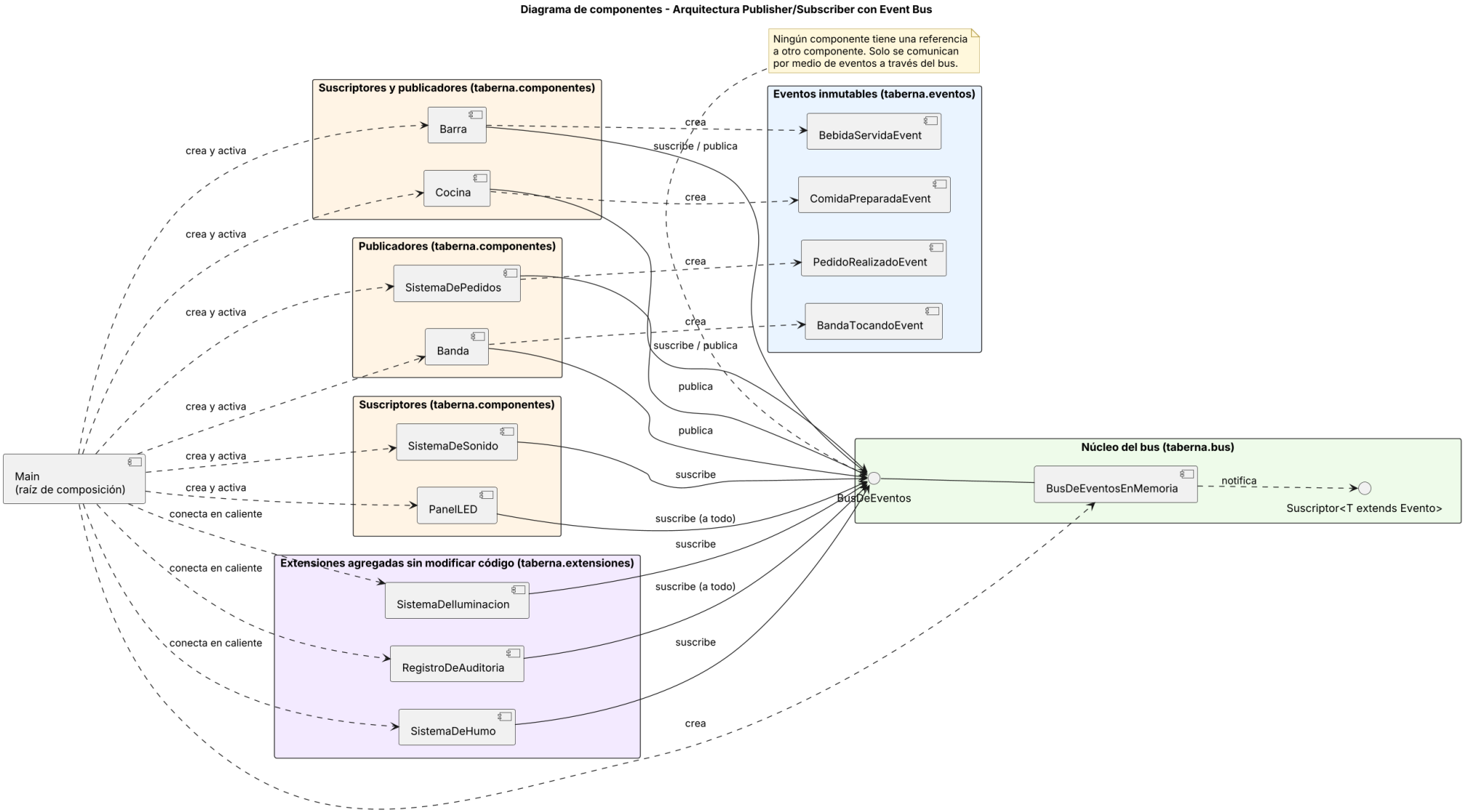
```
deactivate Banda

== Extensibilidad: la banda toca "Through the Fire and Flames" ==

Banda -> Bus : publicar(BandaTocandoEvent\n("Los Seguidores de Dijkstra", "Through the Fire and Flames", 444))
activate Bus
Bus -> Sonido : alRecibirEvento(BandaTocandoEvent)
Sonido -> Sonido : imprime "Ajustando ecualizador para:\nThrough the Fire and Flames"
Bus -> Humo : alRecibirEvento(BandaTocandoEvent)
Humo -> Humo : imprime "¡Canción épica! Disparando efectos de humo"
Bus -> Panel : alRecibirEvento(BandaTocandoEvent)
Panel -> Panel : imprime "Ahora suena: ... Through the Fire and Flames"
deactivate Bus

note over Banda, Humo
  SistemaDeHumo se conectó al bus con suscribir(BandaTocandoEvent.class, this)
  sin modificar ni un carácter de Banda ni de BusDeEventosEnMemoria.
  (SistemaDeIluminacion y RegistroDeAuditoria se conectan igual; se omiten aquí por claridad.)
end note
@enduml
```

4. Diagrama de componentes



Código PlantUML - diagrama de componentes

```
@startuml 03-diagrama-de-componentes
title Diagrama de componentes - Arquitectura Publisher/Subscriber con Event Bus

skinparam dpi 130
skinparam shadowing false
skinparam componentStyle uml2
skinparam packageStyle rectangle
skinparam noteBackgroundColor #FFF8DC
skinparam noteBorderColor #B8A04A
left to right direction

package "Núcleo del bus (taberna.bus)" #EEFBEA {
  interface "BusDeEventos" as IBus
  interface "Suscriptor<T extends Evento>" as ISuscriptor
  component [BusDeEventosEnMemoria] as Bus
}

package "Eventos inmutables (taberna.eventos)" #EAF4FF {
  component [PedidoRealizadoEvent] as EvPedido
  component [BebidaServidaEvent] as EvBebida
  component [ComidaPreparadaEvent] as EvComida
  component [BandaTocandoEvent] as EvBanda
}

package "Publicadores (taberna.componentes)" #FFF4E5 {
  component [SistemaDePedidos] as Pedidos
  component [Banda] as Banda
}

package "Suscriptores y publicadores (taberna.componentes)" #FFF4E5 {
  component [Barra] as Barra
  component [Cocina] as Cocina
}

package "Suscriptores (taberna.componentes)" #FFF4E5 {
  component [SistemaDeSonido] as Sonido
  component [PanelLED] as Panel
}

package "Extensiones agregadas sin modificar código (taberna.extensiones)" #F3EAFF {
  component [SistemaDeHumo] as Humo
  component [SistemaDeIluminacion] as Luces
  component [RegistroDeAuditoria] as Auditoria
}

component [Main\n(raíz de composición)] as Main

Bus -up- IBus
Bus ..> ISuscriptor : notifica

' Todos dependen SOLO de la interfaz del bus
Pedidos --> IBus : publica
Banda --> IBus : publica
Barra --> IBus : suscribe / publica
Cocina --> IBus : suscribe / publica
Sonido --> IBus : suscribe
Panel --> IBus : suscribe (a todo)
Humo --> IBus : suscribe
Luces --> IBus : suscribe
Auditoria --> IBus : suscribe (a todo)

' Qué evento viaja por el bus
Pedidos ..> EvPedido : crea
Barra ..> EvBebida : crea
Cocina ..> EvComida : crea
Banda ..> EvBanda : crea

Main ..> Bus : crea
Main ..> Pedidos : crea y activa
Main ..> Banda : crea y activa
Main ..> Barra : crea y activa
Main ..> Cocina : crea y activa
Main ..> Sonido : crea y activa
Main ..> Panel : crea y activa
Main ..> Humo : conecta en caliente
Main ..> Luces : conecta en caliente
Main ..> Auditoria : conecta en caliente

note bottom of IBus
  Ningún componente tiene una referencia
  a otro componente. Solo se comunican
  por medio de eventos a través del bus.
end note
@enduml
```